

Advanced programming Assignment

ROLL NO: CSB24065

NAME: Gourisankar Bhuyan

Session: Spring

Year:2026

Question:

Assignment 19: Simple Digital Counter & Theme Toggle App Build a single-screen mobile application using React Native. The app functions as a digital counter that allows users to increment, decrement, and reset a number displayed on the screen. To make the app more interactive, it must also include a "Theme Toggle" button that switches the screen's background and text colors between a Light Mode and a Dark Mode. This assignment focuses on your ability to set up a basic React Native environment, layout components cleanly using Flexbox, and manage UI changes dynamically using React's state management.

Implementation Rules

Core Layout: The application must use standard React Native components: View, Text, and TouchableOpacity (or Button). The counter UI should be perfectly centered on the screen.

State Management: Use the useState hook to manage two pieces of state: the current counter value (integer) and the active theme mode (boolean or string).

Counter Logic:

- * The counter should start at 0. The "Increment" button must increase the count by 1. The "Decrement" button must decrease the count by 1, but it should never let the counter go below 0 (prevent negative numbers). The "Reset" button must bring the count back to 0.

Dynamic Styling:

- * **Light Mode (Default):** White background with dark text.
- * **Dark Mode:** Dark gray/black background with white text. Clicking the "Toggle Theme" button should instantly swap these styles across the entire screen. You must have the followings:

1. **UI Layout & Component Structure:** Correctly structure the app using a parent container, a text display for the counter, and a clean arrangement of buttons using Flexbox (e.g., placing the increment/decrement buttons side-by-side). Use proper React Native style properties (flex, justifyContent, alignItems, fontSize, padding).
2. **Counter State & Validation Logic:** Successfully implement the useState hook to track and dynamically display the counter value. Implement the increase, decrease, and reset functions correctly.
3. **Constraint Check:** Add an internal conditional check to ensure that clicking decrement at 0 does nothing, keeping the app safe from negative values.

Dynamic Theme Toggling: Implement state tracking for the theme (e.g., `isDarkMode`). Use conditional styling or ternary operators within your style objects to alter the `backgroundColor` of the main container and the color of the text components based on the theme state.

4. Code Cleanliness & Best Practices: Maintain well-organized code with proper component separation or readable inline styling. Use meaningful variable and function names (e.g., `handleIncrement`, `toggleTheme`). Ensure no obvious runtime crashes occur during interactions. Those who are having android mobile must run it in the mobile in development mode. Those having iphone may run in emulator of android studio.

Answer: code in app.js:

```
import React, { useState } from 'react';
import {
  View,
  Text,
  TouchableOpacity,
  StyleSheet,
  SafeAreaView,
  StatusBar,
} from 'react-native';

export default function App() {
  const [count, setCount] = useState(0);
  const [isDarkMode, setIsDarkMode] = useState(false);

  // Counter Logic
  const handleIncrement = () => {
    setCount(prevCount => prevCount + 1);
  };

  const handleDecrement = () => {
```

```
if (count > 0) {
  setCount(prevCount => prevCount - 1);
}
};
```

```
const handleReset = () => {
  setCount(0);
};
```

```
const toggleTheme = () => {
  setIsDarkMode(prev => !prev);
};
```

// Dynamic theme styles

```
const theme = isDarkMode ? styles.dark : styles.light;
```

```
return (
  <SafeAreaView style={[styles.safeArea, theme.container]}>
    <StatusBar
      barStyle={isDarkMode ? 'light-content' : 'dark-content'}
      backgroundColor={isDarkMode ? '#1a1a2e' : '#f0f4ff'}
    />

    <View style={[styles.container, theme.container]}>

      {/* Header */}
      <View style={styles.header}>
        <Text style={[styles.appTitle, theme.text]}>Counter</Text>
        <Text style={[styles.appSubtitle, theme.subText]}>
          {isDarkMode ? '🌙 Dark Mode' : '☀️ Light Mode'}
        </Text>
      </View>
    </View>
  </SafeAreaView>
);
```

</View>

{/ Counter Display */}*

<View style={[[styles.counterCard, theme.card]]}>

<Text style={[[styles.counterLabel, theme.subText]]}>CURRENT
COUNT</Text>

<Text style={[[styles.counterValue, theme.text]]}>{count}</Text>

<View style={styles.statusDot}>

<View style={

styles.dot,

{ backgroundColor: count === 0 ? '#94a3b8' : '#22c55e' }

]} />

<Text style={[[styles.statusText, theme.subText]]}>

{count === 0 ? 'at zero' : count > 0 ? 'counting up' : ''}

</Text>

</View>

</View>

{/ Increment / Decrement Buttons */}*

<View style={styles.primaryButtons}>

<TouchableOpacity

style={[[styles.actionButton, styles.decrementButton, count === 0 &&
styles.disabledButton]]}

onPress={handleDecrement}

activeOpacity={count === 0 ? 1 : 0.75}

>

<Text style={styles.actionButtonText}>--</Text>

<Text style={styles.actionButtonLabel}>Decrement</Text>

</TouchableOpacity>

<TouchableOpacity

```

    style={[styles.actionButton, styles.incrementButton]}
    onPress={handleIncrement}
    activeOpacity={0.75}
  >
    <Text style={styles.actionButtonText}>+</Text>
    <Text style={styles.actionButtonLabel}>Increment</Text>
  </TouchableOpacity>
</View>

{/* Reset Button */}
<TouchableOpacity
  style={[styles.resetButton, theme.resetBtn]}
  onPress={handleReset}
  activeOpacity={0.75}
>
  <Text style={[styles.resetButtonText, theme.resetText]}>↺ Reset to
Zero</Text>
</TouchableOpacity>

{/* Theme Toggle */}
<TouchableOpacity
  style={[styles.themeButton, theme.themeBtn]}
  onPress={toggleTheme}
  activeOpacity={0.8}
>
  <Text style={[styles.themeButtonText, theme.themeBtnText]}>
    {isDarkMode ? '☼ Switch to Light Mode' : '🌙 Switch to Dark Mode'}
  </Text>
</TouchableOpacity>

</View>

```

```
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
  },
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    paddingHorizontal: 24,
    paddingVertical: 32,
  },

  // Header
  header: {
    alignItems: 'center',
    marginBottom: 40,
  },
  appTitle: {
    fontSize: 36,
    fontWeight: '800',
    letterSpacing: 1.5,
  },
  appSubtitle: {
    fontSize: 14,
    marginTop: 4,
    letterSpacing: 0.5,
  },
});
```

// Counter Card

```
counterCard: {  
  width: '100%',  
  borderRadius: 24,  
  paddingVertical: 40,  
  paddingHorizontal: 32,  
  alignItems: 'center',  
  marginBottom: 32,  
  shadowColor: '#000',  
  shadowOffset: { width: 0, height: 4 },  
  shadowOpacity: 0.12,  
  shadowRadius: 12,  
  elevation: 6,  
},  
counterLabel: {  
  fontSize: 11,  
  fontWeight: '700',  
  letterSpacing: 3,  
  marginBottom: 8,  
},  
counterValue: {  
  fontSize: 88,  
  fontWeight: '900',  
  lineHeight: 96,  
},  
statusDot: {  
  flexDirection: 'row',  
  alignItems: 'center',  
  marginTop: 12,  
  gap: 6,
```

```
},
dot: {
  width: 8,
  height: 8,
  borderRadius: 4,
},
statusText: {
  fontSize: 12,
  fontWeight: '500',
  marginLeft: 6,
},

// Primary Buttons (Increment/Decrement)
primaryButtons: {
  flexDirection: 'row',
  gap: 16,
  marginBottom: 16,
  width: '100%',
},
actionButton: {
  flex: 1,
  borderRadius: 18,
  paddingVertical: 20,
  alignItems: 'center',
  justifyContent: 'center',
  shadowColor: '#000',
  shadowOffset: { width: 0, height: 3 },
  shadowOpacity: 0.15,
  shadowRadius: 8,
  elevation: 4,
},
```



```
incrementButton: {
  backgroundColor: '#3b82f6',
},
decrementButton: {
  backgroundColor: '#ef4444',
},
disabledButton: {
  backgroundColor: '#94a3b8',
  opacity: 0.6,
},
actionButtonText: {
  fontSize: 32,
  fontWeight: '700',
  color: 'ffffff',
  lineHeight: 36,
},
actionButtonLabel: {
  fontSize: 11,
  fontWeight: '600',
  color: 'rgba(255,255,255,0.85)',
  letterSpacing: 0.5,
  marginTop: 2,
},
```

// Reset Button

```
resetButton: {
  width: '100%',
  borderRadius: 16,
  paddingVertical: 16,
  alignItems: 'center',
  marginBottom: 16,
```

```
    borderWidth: 2,  
  },  
  resetButtonText: {  
    fontSize: 15,  
    fontWeight: '700',  
    letterSpacing: 0.3,  
  },
```

// Theme Toggle Button

```
themeButton: {  
  width: '100%',  
  borderRadius: 16,  
  paddingVertical: 16,  
  alignItems: 'center',  
},  
themeButtonText: {  
  fontSize: 15,  
  fontWeight: '700',  
  letterSpacing: 0.3,  
},
```

// — THEMES —

```
light: {  
  container: {  
    backgroundColor: '#f0f4ff',  
  },  
  text: {  
    color: '#0f172a',  
  },  
  subText: {  
    color: '#64748b',
```

```
},
card: {
  backgroundColor: '#ffffff',
},
resetBtn: {
  borderColor: '#0f172a',
  backgroundColor: 'transparent',
},
resetText: {
  color: '#0f172a',
},
themeBtn: {
  backgroundColor: '#0f172a',
},
themeBtnText: {
  color: '#ffffff',
},
},
```

```
dark: {
  container: {
    backgroundColor: '#1a1a2e',
  },
  text: {
    color: '#f1f5f9',
  },
  subText: {
    color: '#94a3b8',
  },
  card: {
    backgroundColor: '#16213e',
```

```
},
resetBtn: {
  borderColor: '#f1f5f9',
  backgroundColor: 'transparent',
},
resetText: {
  color: '#f1f5f9',
},
themeBtn: {
  backgroundColor: '#f1f5f9',
},
themeBtnText: {
  color: '#0f172a',
},
},
});
```